

VDR-LLM-Prolog: Self-Extending Architecture

Registry: [?]

Series Path: [HOWL-VDR-1-2026] → [HOWL-VDR-2-2026] → [HOWL-MATH-3-2026] → [HOWL-MATH-4-2026] → ... → [HOWL-VDR-15-2026] → [HOWL-VDR-16-2026] → [HOWL-VDR-17-2026]

→ [HOWL-VDR-18-2026] → [?]

DOI: 10.5281/zenodo.zzz

Date: May 2026

Domain: Applied Philosophy / Computational Linguistics

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Opus 4.6.

1. What Self-Extending Means

A conventional large language model generates every token of its response through token prediction — arithmetic, formatting, state tracking, hedging, reasoning, and the actual content the user wanted. VDR is a hybrid architecture that separates these concerns. The LLM generates only judgment tokens (deciding what to do) and prose tokens (natural language for human consumption), while exact integer primitives handle computation, state management, formatting, and logical deduction. Data lives in knowledge bases at integer addresses. The LLM references data by typed paths and integer identifiers — it never ingests data through its token stream to process it.

In this architecture the LLM emits structured command tokens, roughly eight tokens per invocation, that call specific primitives on specific data at specific paths. A command token vocabulary of around 300 primitive names and 200 paths gives approximately six bits of entropy per token, compared to the fifteen or so bits per token when generating from full vocabulary. This makes command generation low-entropy, low-error, and cheap.

Self-extending means the system’s knowledge base, rule set, and executable capabilities grow as a natural byproduct of doing work. When the LLM investigates an SRE incident, it doesn’t just produce a report and discard its working state. It writes Prolog rules encoding the correlation patterns it discovered. It writes Python scripts for the analysis it performed. It stores findings as provenanced facts in project knowledge bases. All of these persist, compose with prior accumulated state, and are available to every future session that has scope access to that project.

This accumulation has properties that distinguish it from conventional training. It is immediate — one fact assertion and the knowledge is live, no batch process or gradient computation required. It is inspectable — every rule is a readable Prolog clause, every

fact carries provenance identifying its source, time, operation, and precision. It is reversible — retracting a fact or rule removes it cleanly, unlike weight-based training where bad data poisons the model permanently. It is scoped — knowledge accumulated in one project does not leak to another because the same visibility and scope mechanisms that govern all data access govern self-generated knowledge. And it is incremental — the five hundredth document builds on the rule base from the first four hundred ninety-nine without catastrophic forgetting, because facts sit at integer addresses rather than being distributed across weight matrices.

The result is a system where usage is training. Every session extends the system’s capability, and that extension is permanent, auditable, and composable. The conventional distinction between training time and inference time dissolves.

2. The Compaction Format

When a conventional LLM produces a response, eighty to ninety-five percent of the tokens are infrastructure — formatting, hedging, state reconstruction, arithmetic shown as work, confidence language with no computational basis. The actual information content occupies a small fraction of the token budget.

Compaction is the process of stripping infrastructure tokens while preserving all named entities, their properties, and their relationships in a structured tabular form. A compacted document uses pipe-delimited tables with typed columns, ID-prefixed rows for cross-referencing, explicit relationship tables declaring how entities connect, decode legends explaining the ID scheme, and section indexes mapping content to ID ranges. This format preserves one hundred percent of the informational content at a fraction of the token cost.

The compacted form is structurally close to what VDR stores natively. Each row in a pipe-delimited table is a fact — a predicate with typed fields. Each entry in a relationship table is a Prolog rule — a declared connection between two identified entities with a named relationship type. Each table grouped by a common prefix is a predicate-major column group, which maps directly to VDR’s columnar storage format where all facts sharing a predicate are stored contiguously for parallel access.

The adjustment needed for VDR-19 is to align the compaction syntax with three targets simultaneously. First, Prolog clause syntax, so relationship declarations parse directly into executable rules. Second, the predicate-major columnar layout used by VDR’s GPU-accelerated knowledge base operations, so tables parse directly into the storage format. Third, struct definitions in implementation languages, so table schemas map directly to Zig structs or Python dataclasses.

Consider a concrete example. In the current compaction format, a principle might appear as:

P1|Safety is consequence not feature|no safety-specific modules; safety emerges from KB visibility, grants, and grammar output validation

In the adjusted format targeting Prolog-compatible syntax, this becomes a fact with explicit predicate, typed fields, and a form that parses directly into a KB assertion:

principle(p1, “Safety is consequence not feature”, “no safety-specific modules; safety emerges from KB visibility, grants, and grammar output validation”).

A relationship entry like:

P1|emerges_from|VIS1,GR1,OC1

becomes a set of Prolog rules:

emerges_from(p1, vis1).

emerges_from(p1, gr1).

emerges_from(p1, oc1).

These are directly loadable. The parse step is mechanical — split on delimiters, map prefixed IDs to integer addresses, assert facts into the appropriate predicate-major column group in the target knowledge base. No transformation logic, no interpretation, no LLM involvement. The compacted document is a load file.

3. The Bootstrap Pipeline

The system needs an initial seed before it can self-extend. Critically, the seed is not domain knowledge. It is operational competence — the ability to read input, write output, parse data formats, compose primitives, and manage its own structures. Domain knowledge accumulates through usage after the bootstrap.

Seed Layer 1: Language

The foundation is the ability to read and produce English. This requires a library of sentence structure templates — syntactic patterns covering the range of structures used in working prose. Subject-verb-object, conditional-clause-main-clause, relative clause embedding, participial phrases, and so on. The number of distinct structures used even in good prose is bounded; a library of several thousand covers the practical space generously.

With sentence templates, the LLM’s output generation changes. Instead of predicting every token from full vocabulary, it emits a semantic tuple — subject, verb, object, modifiers, weight criteria — as a handful of command tokens. Prolog rules match against the template library based on semantic roles and weighting criteria. The matching template provides all structural tokens: articles, prepositions, conjunctions, punctuation. Content words from the LLM’s semantic tuple fill typed slots. The output is grammatically correct by construction.

The seed also includes a typo correction knowledge base mapping common misspellings and input errors to corrections, and a classification knowledge base mapping input patterns to tags for session scoring and routing. Together these enable the system to read imperfect user input and generate clean, structured output from its first interaction.

Seed Layer 2: Format Handling

The system needs to ingest and produce structured data. The seed includes parsing and generation grammars for standard formats: JSON, CSV, markdown, and the adjusted compaction format itself. Each grammar specifies the structural tokens for the format — braces, brackets, commas, pipes, headers — and the typed content slots that the LLM or primitives fill with actual values.

A grammar, once loaded, handles all future documents of that format without LLM involvement. The structural tokens are emitted by the grammar engine, not predicted by the LLM. This eliminates formatting errors (mismatched braces, missing commas, unescaped characters) by construction and reduces token cost by thirty to sixty percent depending on format.

Seed Layer 3: Operational Environment

The primitives exist as executable code, but the system needs knowledge about when and how to compose them. Seed layer three is a knowledge base of operational rules: which builtin to call for which task, how to sequence pipeline stages, when to write a Prolog rule versus store a flat fact, how to manage data structures — when to use a queue versus a ring buffer, how to check and drain queues, when to increment counters, how to use LRU caches for frequently accessed values.

This layer also includes the compaction rules themselves — how to identify named entities, relationships, and structure in incoming documents and map them to KB facts and Prolog rules. These rules enable the system to process new documents into its native storage format.

Seed Layer 4: Self-Maintenance

Rules for recognizing when a new grammar is needed for a novel document structure. Rules for detecting when existing compaction patterns don't cover an incoming document type. Rules for project lifecycle management — when to snapshot state, how to compare versions, when to promote session findings to project-level knowledge.

The Bootstrap Sequence

The bootstrap proceeds through stages with clear transition criteria.

In the first stage, a conventional LLM compacts seed documents into the adjusted format. These compacted documents are parsed through primitives into knowledge bases — named entities become facts at integer addresses, relationships become Prolog rules, tables become predicate-major column groups. The system now has operational competence.

In the second stage, the system begins operating. Users interact with it, feed it documents, ask it to investigate problems. At this point, new documents that need compaction still go through an external conventional LLM, because the system's compaction rule base is thin.

In the third stage, through normal usage the system has accumulated enough compaction grammars, structural patterns, and domain classification rules that it can compact new documents itself. The LLM's judgment identifies named entities and relationships. Primitives parse. Prolog rules classify. Grammars format. The conventional LLM is no longer needed as a preprocessing step.

From this point forward, the system sources data directly — web fetch, paste, upload, API — and compacts, stores, and indexes it through its own accumulated capabilities. Each new document potentially extends the rule base further. The bootstrap is complete and continuous self-extension has begun.

4. The Operational Lifecycle

Once bootstrapped, the system operates in a continuous cycle with four phases that blend together in practice.

Intake

Documents enter through any supported channel — web fetch with credentials, file upload, paste, API ingestion. The system compacts incoming documents using its accumulated grammars and classification rules. Named entities are identified, relationships are extracted, structure is mapped to KB facts and Prolog rules. Every ingested fact carries full provenance: source, timestamp, original format, conversion method, and precision where applicable.

The data never enters the LLM's token stream during intake. Primitives handle parsing, the grammar engine handles structural recognition, Prolog rules handle classification. The LLM is involved only if judgment is required — resolving ambiguity in entity identification, deciding how to categorize a novel relationship type, flagging a contradiction with existing knowledge.

Processing

When the LLM or a user queries the accumulated knowledge base, existing Prolog rules fire automatically against the stored facts. If new documents have been ingested since the last query, rules that were written during prior investigations evaluate against the new facts without any additional LLM involvement. Contradictions between new and existing facts surface through rule evaluation. Confirmations strengthen confidence values through declared propagation rules.

The LLM's role during processing is judgment — interpreting what the rule evaluations found, deciding what matters, identifying what needs further investigation, assessing whether the accumulated findings answer the user's question.

Rule Generation

As the LLM works, it writes new Prolog rules encoding patterns it discovers. A correlation between deployment timestamps and error rate spikes becomes a rule. A classification

pattern for a type of legal clause becomes a rule. A relationship between drug interactions becomes a rule. Each rule is asserted into the appropriate project knowledge base through the standard command token pipeline — the LLM emits roughly eight tokens, the primitive executes, the rule is live and queryable immediately.

When analysis requires procedural logic beyond what Prolog rules express naturally — statistical computations, complex transformations, custom visualizations — the LLM writes a Python script. The script executes in a sandboxed Docker container with grant-gated filesystem access. Results return as typed values stored in the knowledge base with provenance linking them to the script, its inputs, and its execution context.

The LLM also writes new grammars when it encounters document structures not covered by existing grammars, and new compaction rules when it encounters document types whose entity and relationship patterns aren't covered by existing compaction rules.

Accumulation

Facts, rules, scripts, grammars, and project state accumulate within scope. Session-level state — working hypotheses, intermediate findings, exploration paths — lives in the session knowledge base and is discarded when the session ends unless the LLM makes an explicit judgment to promote specific findings to project-level knowledge. This is an important filter: not everything discovered in a session is worth keeping permanently. The LLM's judgment about what to promote is itself a form of curation that improves the quality of accumulated knowledge.

Project-level state persists across sessions and is available to all future sessions with scope access. At meaningful points the system snapshots project state. Comparison rules, themselves stored as Prolog, can diff current state against prior snapshots. The system knows what changed, when, and why, because every fact carries provenance.

5. The LLM as Runtime Programmer

The LLM in a VDR system is not a chatbot that generates text responses. It is a programmer that extends the runtime it operates within. This distinction is central to understanding self-extension.

Writing Prolog Rules

When the LLM asserts a Prolog rule, that rule becomes immediately available to VDR's frontier-based GPU evaluator. The evaluator transforms recursive depth-first Prolog search into batched joins borrowed from GPU database query processing — candidate retrieval, filtering, unification, and body goal joining — achieving high throughput on parallel hardware. A rule written by the LLM during an SRE investigation composes automatically with all existing rules and facts within scope. If a new document is ingested next week containing facts that match the rule's head, the rule fires without any LLM involvement.

The economics of rule writing favor doing it aggressively. A Prolog rule costs roughly twenty-five to forty tokens to formalize and assert. On first use it replaces what would have been one hundred fifty to three hundred tokens of conventional LLM reasoning. By the fifth use the amortized cost is negligible. Rules at organizational scope, shared across all projects in a department, can be reused thousands of times — the per-use cost approaches zero. Every rule written is an investment that pays returns on all future work within scope.

Writing Python Scripts

Some analysis requires procedural logic that Prolog doesn't express naturally. The LLM writes a Python script — typically twenty to fifty tokens of actual judgment about what computation to perform — and the system executes it in a sandboxed Docker container. The script has access only to data paths that the session's grants authorize. Results return as typed values with provenance and are stored in the knowledge base.

Critically, the script persists. When similar analysis is needed in a future session, the LLM can re-execute the existing script on new data rather than writing a new one. The script becomes infrastructure — a reusable analytical capability that the system accumulated through usage.

Writing Grammars

When the LLM encounters a novel document structure — a new log format, a new report template, a new data export format — it can write a grammar for that structure. The grammar specifies the structural tokens and typed content slots. Once stored, the grammar handles all future documents of that type at the primitive level without LLM involvement. The LLM's one-time judgment about the document's structure becomes a permanent parsing capability.

Writing Compaction Rules

As the system processes more documents from more domains, the LLM writes rules about how to compact specific document types. Which entities to extract from a medical paper. Which relationships to encode from a legal contract. Which structure to preserve from an API specification. These rules enable self-compaction of future documents of the same type, closing the loop from the bootstrap pipeline — the system that once needed an external LLM for compaction now compacts autonomously.

6. The SRE Operational Environment

Site reliability engineering provides a concrete demonstration of self-extending architecture because SRE work is repetitive in structure but variable in specifics. The same categories of investigation recur — performance degradation, deployment correlation, resource exhaustion, dependency failures — but each incident has different metrics, different services, different root causes. This is precisely the pattern that self-extension exploits: accumulate structural knowledge, apply LLM judgment only to what's genuinely novel.

First Investigation

A new system with only the bootstrap seed. An SRE engineer reports elevated error rates.

The LLM queries the Prometheus API using credentialed, positionally constrained command tokens. The response — potentially megabytes of time-series metrics — routes directly to a knowledge base target through primitives. The LLM never sees the raw data. It receives a typed summary: how many series returned, time range, approximate volume.

The LLM emits command tokens to parse the JSON response, filter by error rate threshold, sort by severity. All primitive operations, all exact, all on data at integer addresses. The LLM sees the filtered result set as typed facts.

The LLM writes a Python script to compute correlation between deployment timestamps and error rate changes. The script executes in a Docker sandbox against the filtered data. Results return as typed values: three services show strong correlation, deployment occurred forty-seven minutes before error spike.

The LLM writes a Prolog rule: `deployment_correlated(Service, Deployment)` when `error_rate_change(Service, Change)` exceeds threshold and `deployment_time(Deployment, Time)` falls within correlation window of `Change`.

The LLM stores all findings as provenanced facts in the project knowledge base. It generates a formatted incident report through grammar templates, exports CSV and JSON artifacts. It issues a credentialed service restart on the most affected service.

Total LLM tokens for the entire investigation: roughly two hundred, almost all judgment and command tokens. Total data processed through primitives: megabytes. Time: seconds, not minutes.

Second Investigation

Same system, new incident. The Prometheus data comes in and is parsed and filtered as before. But now the deployment correlation rule from the first investigation fires automatically against the new data. The LLM doesn't need to hypothesize about deployment correlation — the rule already checks it and reports whether a match exists.

The Python correlation script already exists. The LLM re-executes it on the new data rather than writing a new one.

The filtering patterns are already in the operational knowledge base. The grammar templates for the incident report are already stored.

The LLM's judgment focuses on what's different about this incident. Maybe the correlation is weaker. Maybe a new failure pattern has emerged. The LLM writes a new rule for the new pattern and stores new findings. The knowledge base grows.

Token cost is roughly forty-two percent lower than the first investigation because accumulated rules and scripts handle the known patterns automatically.

Tenth Investigation

The project knowledge base now contains comprehensive correlation rules, filtering patterns, analysis scripts, and findings from nine prior incidents. Common failure patterns are largely handled by rule evaluation without LLM involvement. The LLM's judgment is reserved for genuinely novel aspects of each new incident.

A new engineer joining the team inherits all accumulated rules, scripts, and findings through scope access. They don't need to rebuild investigation methodology from scratch. The system's accumulated knowledge is their starting point.

Hundredth Investigation

Routine triage is substantially automated. Metrics come in, rules fire, known patterns are identified and reported. The LLM handles exceptions — patterns that don't match existing rules, novel failure modes, unusual combinations. When it encounters something new, it writes new rules, extending the system further.

The project knowledge base at this point is a queryable, provenanced, inspectable model of the operational domain — not in neural network weights, but in explicit facts and rules. Any finding can be traced to its source data. Any rule can be examined for correctness. Any conclusion can be reproduced deterministically.

7. Data Flow Architecture

In a conventional LLM system, data enters the context window as tokens. The LLM processes data through attention — comparing each token position against all other positions to determine relevance. Results are generated as tokens. State is lost between turns unless the entire conversation history is re-read, which is why conventional cost scales quadratically with conversation length.

In VDR, data lives at integer addresses in knowledge bases, queues, LRU caches, ring buffers, counters, and stacks. The LLM references data by dotted path names and integer identifiers. It never ingests data to manipulate it.

The data flow for a typical operation: external data enters through primitives (API fetch, file read, document parse) and is stored as facts at integer addresses in a knowledge base. The LLM emits command tokens referencing the path where data was stored. Primitives operate on the data at those addresses — filtering, sorting, aggregating, comparing. Results are written to new addresses in the knowledge base. The LLM receives a typed summary of results. Output is generated through grammar templates that pull values from knowledge base addresses into formatted slots.

At no point does the raw data flow through the LLM's token stream. The LLM processes references, not content. This is why VDR can handle data volumes that conventional LLMs structurally cannot — a one megabyte JSON response, a ten megabyte document, a five hundred position portfolio. The data never needs to fit in a context window because it never enters one.

Queue-Based Multi-Instance Orchestration

The data flow architecture enables a topology beyond single-user single-LLM conversation. An LLM instance can write findings to a knowledge base and put a typed summary on a queue. Another instance — a fresh clone with full access to accumulated project state — picks up from the queue, reads the provenance, continues the work.

Each instance stays fresh. Clone economics from VDR-15 show that each disposable clone operates within its optimal range (early conversation, no attention degradation) while inheriting all accumulated knowledge at integer addresses. The clone lifecycle costs roughly forty tokens — a snapshot, a spawn, checks, and eventually a kill and respawn. Knowledge accumulates across instances while each individual LLM stays permanently at peak capability.

This enables horizontal scaling of self-extension. One instance handles data acquisition and initial filtering. Another handles analysis. A third handles synthesis and report generation. Each writes rules and findings that the others can access through shared project knowledge bases. The system extends itself in parallel.

8. Train-As-You-Go

Conventional machine learning draws a sharp line between training and inference. Training is a batch process: collect data, compute gradients, update weights, evaluate, deploy. Inference is a separate phase where the trained model processes inputs. Improving the model means retraining — expensive, slow, and disruptive.

In VDR the distinction dissolves. Every document ingested becomes queryable facts. Every investigation writes reusable rules. Every novel structure generates a persistent grammar. The system's knowledge and inference capability grow continuously through normal usage. There is no separate training phase because usage is training.

This has properties that weight-based training cannot provide.

It is immediate. Asserting a fact or rule into a knowledge base is one primitive call costing eight command tokens. The knowledge is live and queryable in the same turn it was created. No batch process, no gradient computation, no retraining cycle, no deployment.

It is inspectable. Every Prolog rule is a readable clause that a human can examine, understand, and verify. Every fact carries provenance — source, timestamp, operation, conversion method, precision. The entire learned state of the system is transparent. There is no equivalent of examining neural network weights to understand what a model learned.

It is reversible. Retracting a fact or rule removes it cleanly from the knowledge base. Its consequences disappear from future queries. There is no equivalent of the weight poisoning problem where bad training data permanently degrades model capability in ways that are difficult to identify and impossible to surgically remove.

It is scoped. Knowledge accumulated in one project is governed by the same visibility and scope mechanisms that govern all data access. A rule written in a chemical engineering

project cannot be queried from a marketing project unless scope explicitly permits it. Knowledge isolation is structural, not policy-based.

It is incremental. Each new document extends the existing knowledge base without displacing prior knowledge. There is no catastrophic forgetting because facts are stored at stable integer addresses, not distributed across weight matrices where new learning can overwrite old representations.

It is auditable. Every fact carries provenance. Every rule carries provenance. Every modification is logged in an append-only audit knowledge base. A compliance officer can trace any conclusion to its source data, through every intermediate derivation, and verify that each step was authorized and correct.

It is composable. Prolog rules from different sources — different sessions, different users, different projects within scope — interact through structural unification automatically. A rule about deployment correlation written during one investigation composes with a rule about resource utilization written during another, producing inferences that neither rule could produce alone, without any explicit integration step.

The Accumulation Curve

Early in a system's lifecycle, most work is LLM judgment — writing foundational rules, learning the structure of the operational domain, establishing grammars and compaction patterns. The ratio of LLM tokens to useful output is at its highest.

As the system accumulates rules, scripts, grammars, and knowledge, the ratio shifts. Existing rules handle an increasing share of routine operations. Existing scripts re-execute on new data. Existing grammars parse new documents of known types. The LLM's judgment focuses increasingly on genuinely novel situations — new patterns, new contradictions, new categories of problem.

The system does more useful work per LLM token as it accumulates. This is the opposite of the conventional pattern, where each turn costs more (quadratic attention growth) and produces less reliable output (attention degradation over long contexts). In VDR, each session is cheaper than the prior session for similar work, and the output is more reliable because it builds on a more comprehensive verified knowledge base.

9. Capability Growth Model

The system's capability can be quantified along several dimensions that all grow monotonically through usage.

Knowledge base facts measure the raw volume of stored, queryable, provenanced information. An SRE project might accumulate a hundred facts from its first investigation, growing to several hundred by the fifth, approaching a thousand by the twentieth. Each fact is individually queryable at constant time through integer-addressed knowledge base lookup.

Prolog rules measure the system's inference capability — the patterns it can recognize,

the correlations it can detect, the classifications it can perform without LLM involvement. Rules accumulate at project scope (available to this investigation), department scope (available to all SRE investigations), and organization scope (available to all projects). Higher-scope rules amortize more aggressively because they serve more queries.

Python scripts measure procedural analytical capabilities. Each script is a reusable computation that can be re-executed on new data. The system’s library of analytical scripts grows with each novel analysis the LLM performs.

Grammars measure the system’s ability to handle structured data formats. Each grammar enables zero-LLM parsing and generation for a document type. The library of grammars grows as the system encounters new formats.

Compaction rules measure the system’s ability to ingest new documents autonomously. Each compaction rule covers a document type — how to identify entities, extract relationships, map structure to knowledge base facts. As the compaction rule library grows, fewer document types require LLM judgment for ingestion.

Session-over-session token reduction measures the practical economic benefit. Each session doing similar work to a prior session is cheaper because accumulated rules and scripts handle known patterns. The reduction is measurable and grows with the knowledge base.

For an SRE use case, a projected growth trajectory: the first investigation produces roughly a hundred facts, fifteen rules, and three scripts. By the fifth investigation, the project has accumulated around three hundred facts, forty rules, and eight scripts, with measurable token reduction on routine analysis. By the twentieth investigation, roughly eight hundred facts, eighty rules, fifteen scripts, and routine triage is substantially handled by rule evaluation. By the fiftieth investigation, the project has a comprehensive domain knowledge base. Novel incidents still require LLM judgment for genuinely new patterns, but the system’s accumulated capability handles the structural work.

10. Security Properties of Self-Extension

VDR enforces access control structurally through four independent mechanisms. Knowledge base visibility controls (public, internal, or owner-only, checked by integer comparison) ensure unauthorized data never enters the LLM’s context. Scope chains (walking from the user’s position upward through the knowledge base tree, with sibling branches structurally unreachable) limit which knowledge bases are searchable. Grants (default denial on all forty-four operational primitives, with positive credential required for each) gate what operations can be performed. Output constraints (grammar-layer validation post-generation, pattern matching on content slots) catch restricted content before it reaches the user.

Self-extension inherits every one of these properties because it operates through the same mechanisms.

When the LLM writes a Prolog rule, it does so by emitting command tokens that invoke the knowledge base assertion primitive. That primitive checks grants before executing — does this session have write access to this knowledge base path? The asserted rule is

stored in a knowledge base subject to the same visibility and scope controls as every other fact. A rule written in project A cannot be queried from project B unless scope permits, because the scope check runs on every knowledge base query regardless of whether the fact was seeded at bootstrap, ingested from a document, or written by the LLM during operation.

When the LLM writes a Python script, the script executes in a sandboxed Docker container. The container's filesystem access is grant-gated — the script can read and write only the data paths that the session's grants authorize. The script's results are stored in the knowledge base with provenance identifying the script, its inputs, its execution context, and the session that created it.

When the LLM writes a new grammar or compaction rule, these are stored as knowledge base facts with the same visibility, scope, and provenance as any other fact. They are queryable, auditable, and retractable through the same mechanisms.

Self-generated rules carry provenance identifying the session, user, and turn that created them. Any rule can be traced to the context in which it was written. Retraction of self-generated rules follows the same audit trail as any other knowledge base modification — logged in the append-only audit knowledge base, with constraint checks preventing unauthorized retraction.

No new attack surface is created by self-extension. The LLM was already writing to knowledge bases and executing operations through the command token and primitive pipeline. Self-extension is just a pattern of usage — the LLM writes rules and scripts that it or future instances will use. The access control checks are the same checks that run on every operation. The audit trail is the same audit trail that logs every access. The security model doesn't need to be extended to cover self-extension because self-extension is not a new capability — it is the natural use of existing capabilities.

11. Language and Dialect as Knowledge Base Selection

When the LLM generates prose in a conventional system, it predicts every token from full vocabulary — structural words, content words, punctuation, formatting — all through the same token prediction mechanism. The output language and register are properties of the model's training distribution. Producing consistent output in a specific dialect or register requires prompt engineering that the model may not sustain across long documents.

In VDR, the LLM's judgment about what to say is decoupled from how to say it. The LLM emits a semantic tuple as command tokens — subject, verb, object, location, weight criteria. This is language-independent content: the meaning the LLM wants to express, stripped of all linguistic structure.

Prolog rules then match against sentence structure templates in the currently mounted language or dialect knowledge base. The rules unify on semantic roles — which templates accept two subjects, which have a location adjunct, which support the desired emphasis pattern — and apply weighting criteria to rank candidates. The matching template provides every structural token: articles, prepositions, conjunctions, clause connectors, punctuation. Content words from the LLM's semantic tuple fill typed slots.

Switching language or dialect is a scope change — mount a different knowledge base. The same semantic tuple routed through an American English knowledge base, a Southern dialect knowledge base, a formal British English knowledge base, or a French knowledge base produces different output. The LLM emits the same command tokens in every case. No model fine-tuning, no prompt engineering, no risk of the model drifting back to its dominant training distribution after a few paragraphs.

The structural rules in the mounted knowledge base are deterministic, so dialect consistency is guaranteed throughout a document of any length. An author can mix registers intentionally — narrator in formal English, dialogue in regional dialect — by switching the mounted knowledge base per output segment. The LLM never needs to track which voice it is using because scope handles that.

Translation falls out naturally. Same semantic tuple, different language knowledge base, different output language. The quality ceiling is the breadth and accuracy of the sentence template library, not the model’s training distribution for that language.

This capability is itself subject to self-extension. Sentence template knowledge bases are accumulated knowledge like any other — written and refined through usage, scoped and versioned and provenanced, growing as the system encounters more linguistic patterns. A system that starts with a few thousand English templates accumulates more through use, eventually covering the full working range of the language with templates that have been verified through actual output.

Appendix A: Compaction Format Specification

The current compaction format uses pipe-delimited tables with ID prefixes, relationship tables, decode legends, and section indexes. The adjusted format modifies syntax for three-way compatibility: Prolog clause parsing, predicate-major columnar storage, and implementation language struct mapping.

Current format entity row:

P1|Safety is consequence not feature|no safety-specific modules designed

Adjusted format (Prolog-compatible fact):

principle(p1, “Safety is consequence not feature”, “no safety-specific modules designed”).

Current format relationship row:

P1|emerges_from|VIS1,GR1,OC1

Adjusted format (Prolog-compatible rules):

emerges_from(p1, vis1).

emerges_from(p1, gr1).

emerges_from(p1, oc1).

Current format table header:

```
principles(id|principle|detail)
```

Adjusted format (struct/schema definition):

```
:- schema(principle, [id:atom, principle:string, detail:string]).
```

The adjusted format is directly loadable by the parse pipeline: split on clause boundaries, extract predicate name and typed arguments, map atom IDs to integer addresses, assert into the predicate-major column group in the target knowledge base. Schema declarations map to Zig struct definitions and Python dataclass definitions through mechanical code generation.

Appendix B: Seed Layer Contents

Seed layer one (language): sentence structure template library (initial inventory of several thousand patterns covering declarative, interrogative, conditional, relative clause, participial, and coordinated structures), typo correction knowledge base (common misspelling-to-correction mappings), input classification knowledge base (pattern-to-tag mappings for session scoring).

Seed layer two (format handling): parsing and generation grammars for JSON, CSV, markdown, pipe-delimited tables, and the adjusted compaction format. Each grammar specifies structural tokens and typed content slots.

Seed layer three (operational environment): primitive composition rules (which builtins for which tasks, pipeline sequencing, data structure selection), compaction rules (entity identification, relationship extraction, structure mapping for known document types), queue/counter/cache management rules.

Seed layer four (self-maintenance): grammar creation triggers (novel structure detection), compaction rule creation triggers (uncovered document type detection), project lifecycle rules (snapshot criteria, version comparison, session-to-project promotion criteria).

Appendix C: Bootstrap Stage Transitions

Stage one to stage two: all four seed layers loaded and queryable. System can parse input, generate output, invoke primitives through command tokens, and manage basic data structures. Transition criterion: successful execution of a test interaction exercising each seed layer.

Stage two to stage three: system has accumulated enough compaction grammars and classification rules through usage that it can compact documents of types it has previously encountered without external LLM assistance. Transition criterion: successful self-compaction of a test document for each known type, validated against external LLM compaction of the same document.

Stage three onward: continuous self-extension. No further stage transitions — the system grows monotonically through usage. The boundary between bootstrap and normal

operation dissolves.

Appendix D: SRE Worked Example

Three investigations showing accumulation. Investigation one: full pipeline from Prometheus query through analysis to report, writing fifteen rules and three scripts. Investigation two: seven of fifteen prior rules fire automatically, two scripts re-execute on new data, eight new rules written, token cost forty-two percent lower. Investigation ten: comprehensive rule base handles routine triage, LLM judgment reserved for novel patterns, token cost roughly seventy percent lower than investigation one for comparable incidents.

Appendix E: Prolog Rule Amortization in Self-Extension Context

A rule formalized at twenty-five to forty tokens. First use replaces one hundred fifty to three hundred tokens of conventional reasoning. Fifth use: amortized cost under ten tokens. At organizational scope with thousands of reuses: amortized cost approaches zero. Self-extension amplifies amortization because every rule written during any session is available to all future sessions within scope.

Appendix F: Python Script Lifecycle

Write (LLM emits twenty to fifty judgment tokens) → store in project knowledge base with provenance → sandbox execution in Docker with grant-gated filesystem → results to knowledge base as typed provenanced values → re-execution on new data in future sessions (zero LLM tokens, only command tokens to invoke) → modification if needed (LLM writes updated version, prior version retained with provenance).

Appendix G: Queue-Based Multi-Instance Coordination

Instance A writes findings to project knowledge base, pushes typed summary to queue. Instance B (fresh clone with scope access) reads from queue, accesses project knowledge base, continues work. Instance B writes additional findings, pushes to queue. Instance C picks up. Each instance is disposable — fresh LLM, full accumulated knowledge, no degradation. Queue items carry provenance linking to the producing instance, session, and turn.

Appendix H: Capability Growth Metrics

Projected growth curves for SRE use case showing knowledge base facts, Prolog rules, Python scripts, grammars, and compaction rules over fifty investigations. All curves monotonically increasing. Session token cost curve monotonically decreasing for comparable work. Ratio of rule evaluations to LLM judgment tokens increases over time — the system does more per LLM token as it accumulates.

Appendix I: Security Properties of Self-Generated Rules

Every self-generated rule is a knowledge base fact. Every knowledge base fact is subject to visibility checks (integer comparison), scope resolution (ancestor walk), grant verification (set membership), and audit logging (append-only). Self-generated rules carry provenance: session ID, user ID, turn number, source data references. Retraction requires grant authorization and is logged. No mechanism exists to write a rule that bypasses its own knowledge base’s access controls.

Appendix J: Language and Dialect Knowledge Base Structure

Each language or dialect knowledge base contains: sentence structure templates (predicate with semantic role slots, structural token sequence, weight attributes for register/formality/complexity), lexical mappings (semantic concepts to language-specific content words), morphological rules (agreement, conjugation, declension as applicable), and prosodic weight rules (for emphasis and rhythm ranking). Templates are Prolog-queryable facts. Selection is rule-based unification on semantic roles plus weight criteria.

Appendix K: Compaction Rules as Self-Extending Grammar

Each compaction rule specifies: document type signature (how to recognize this type), entity extraction patterns (what to pull out as named entities), relationship extraction patterns (what connections to encode), structure mapping (how to organize extracted content into predicate-major tables), and provenance template (what metadata to attach). Compaction rules are themselves knowledge base facts, written by the LLM during document processing, and accumulate through usage.

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-15-2026](#)] VDR-LLM-Prolog: Prompt Optimization. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-15-2026>

[[HOWL-VDR-16-2026](#)] VDR-LLM-Prolog: Safe by Contract. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-16-2026>

[[HOWL-VDR-17-2026](#)] VDR-LLM-Prolog: Alignment. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-17-2026>

[[HOWL-VDR-18-2026](#)] VDR-LLM-Prolog: Performance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-18-2026>